

Hochschule für angewandte Wissenschaften Hamburg
Automatisierungstechnik,
Verbundprojekt

Geiler Robi macht noch geilere Stunts

Stefo Steffansen, Bent Benjo, Lucy Lukas

Hamburg
2. Juli 2026

Name Stefo Steffansen, Bent Benjo, Lucy Lukas
Matrikelnummer 1111111, 22222222 , 333333333

Hochschule HAW Hamburg
Fakultät Automatisierungstechnik
Studiengang Automatisierungstechnik

Erstprüfer Prof. Dr.-Ing. Maaßi boiii
Zweitprüfer Prof. Dr. KA

Bearbeitungszeitraum Wann genau 2025/2026

Inhaltsverzeichnis

Konventionen

Senkrechte Schrift für Zahlen, Einheiten und Operatoren. Kursive Schrift für mathematische Variablen und physikalische Größen. Ableitungen werden mit einem einfachen Punkt – $\dot{x}$ – über ihnen gekennzeichnet. Der Mittelwert einer Variable wird mit einem Balken über der Variable – $\bar{x}$ – ausgedrückt.

Größe:	Größensymbol:	Einheitenzeichen:
Radgeschwindigkeit	$V_{w1}/V_{w2}/V_{w3}$	m s^{-1}
Robotergergeschwindigkeit	$\dot{x}/\dot{y}$	m s^{-1}
Winkelgeschwindigkeit Roboter	$\dot{\varphi}$	rads^{-1}
Teilungswinkel Kettenrad	α	rad
Relativer Drehwinkel	φ	rad
Absolute Winkelposition	ϕ	rad
Winkelgeschwindigkeit Kettenrad	ω	rad s^{-1}
Bandgeschwindigkeit	v_{Belt}	mm s^{-1}
Strecke	x	m
Wirksamer Durchmesser	d_W	m
Wirksamer Radius	r_W	m
Maximaler Radius	r_S	m
Reeller Radius	r_{rel}	m
Periodendauer	T	s
Aktuelle Zeit	t	s
Frequenz	f	Hz
Polygonfrequenz	f_{poly}	Hz
Abtastrate	f_S	Hz
Zahneingriffsfrequenz	f_Z	Hz
Masse	m	kg
Massenträgheitsmoment	J	kg m^2
Kraft	F	N
Drehmoment	M	N m
Drehmoment Motor	M_{Mot}	N m
Beschleunigungsmoment	M_{Besch}	N m
Anzahl der Zähne	Z	1
Impulse pro Umdrehung	IpD	1
Anzahl Inkremente	Inc	1
Unförmigkeit der Banddynamik	δ	1
Effektivwert	RMS	1
Getriebeübersetzung	i	1
Zähler Harmonische	k	1, 2, 3

Tabelle 1: Konventionen

1 Einleitung

Hier kommt die Einleitung hin. Siehe [**Buch**].

2 Konzept

Das Konzept zur Erfüllung der Aufgaben von Gewerk 3 wurden in enger Zusammenarbeit mit Gewerk 2 erstellt. Schon in den ersten Wochen des Projektes hat sich Gewerk 2 für ein Gradientenverfahren entschieden, um die Bahnplanung und die Kollisionvermeidung zu realisieren. Als Reaktion auf dieses Vorhaben wurden dann konzeptionell zwei Ansätze verfolgt:

Für die freie Fahrt - also alle Bereiche außerhalb der Taschenbereiche - wird von Gewerk 2 ein Geschwindigkeitssollwert und eine Richtung übermittelt. Dieser Sollwert muss von einem Geschwindigkeitsregler umgesetzt werden. Dieser Modus heißt Vektorregelung. Ein Positionsregler wird in diesem Modus nicht benötigt.

Für die Taschenbereiche - also dort wo hohe Präzision verlangt wird- wird von Gewerk 2 ein Positionssollwert übermittelt. Dieser Sollwert muss von einem Positionsregler umgesetzt werden. Dieser Positionsregler generiert basierend auf der aktuellen Position ein Geschwindigkeitssollwert. Dieser wird dann an den Geschwindigkeitsregler weitergeleitet. Dieser Modus heißt Positionregelung.

In beiden Modi wird die aktuelle Position der Roboter durch einen Beobachter geschätzt. Die aktuelle Position wird entweder an Gewerk 2 übermittelt (Vektorregelung) oder mit der Sollposition (Positionregelung) verglichen. Der Beobachter wird von Gewerk 3 implementiert und getunt.

2.1 Blockschaltbild

Hier dann auch die Aufteilung in Unterblöcke besprechen

2.2 Schnittstellen

2.3 Interne Statemaschine der Regelung

Hier dann nochmal die interne Statemaschine der Regelung auflisten

3 Regler

Das Kapitel arbeitet sich von der untersten Reglerebene am Rad bis zum übergeordneten Lageregler vor und folgt damit der Signalkette aus Abbildung Konzept in umgekehrter Richtung. Den Anfang macht der Drehzahlregler, der an jedem der drei Räder dafür sorgt, dass die vorgegebene Solldrehzahl anliegt. Da Gewerk 2 jedoch keine Raddrehzahlen, sondern eine Sollgeschwindigkeit des Roboters liefert, ist eine Transformation von Roboter in Motorkoordinaten nötig. Damit sind alle Bausteine für die Vektorfahrweise beisammen. Für die Positionsfahrweise kommt eine weitere Ebene hinzu: Die gemessene Position liegt in Weltkoordinaten vor und muss in das Roboterkoordinatensystem überführt werden. Darauf setzt schließlich der Lageregler auf, dessen Parameter über ein Gain-Scheduling an die jeweilige Situation angepasst werden, um eine zuverlässige Fahrt sicherzustellen.

3.1 Drehzahlregler

Der Drehzahlregler sorgt dafür, dass eine vorgegebene Sollgeschwindigkeit des Roboters auf die Motorbewegung der einzelnen Motoren übersetzt wird und korrekt umgesetzt wird. Der Drehzahlregler wird in TwinCAT als Funktionsbaustein umgesetzt. Als Eingangsgrößen werden Sollgeschwindigkeiten in X,Y, und θ im Roboterkoordinatensystembenötigt. Außerdem werden die Istgeschwindigkeiten der einzelnen Motoren an den Eingang zurückgeführt. Der Ausgang ist das Geschwindigkeitssignal für die einzelnen drei Motoren. Abbildung 3.1 zeigt den Drehzahlregler als Blackbox mit Eingangs- und Ausgangssignalen.

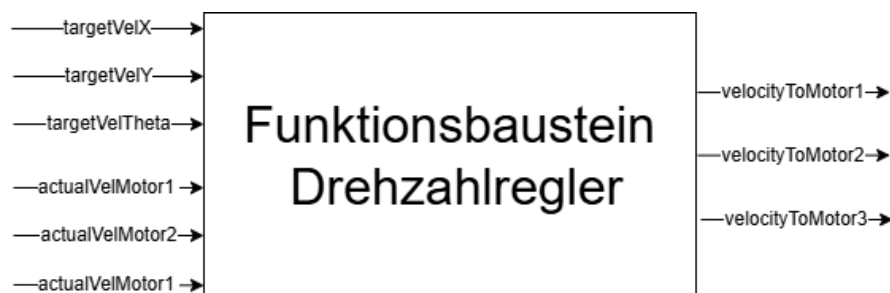


Abbildung 3.1: Drehzahlregler als Blackbox mit Eingangs- und Ausgangssignalen

TargetVelocity

Die Target Velocity ist die Sollgeschwindigkeit des Roboters im Koordinatensystem des Roboters. Sie wird in $\frac{m}{s}$ angegeben und besteht aus drei voneinander unabhängigen

Komponenten: X, Y und θ . Die X-Komponente beschreibt die Geschwindigkeit in Vorwärtsrichtung, die Y-Komponente die Geschwindigkeit in Querrichtung und die θ -Komponente die Winkelgeschwindigkeit um die Z-Achse. Durch Überlagerung der Bewegungen ist der Roboter in der Lage sich in alle Richtungen im Raum zu bewegen.

VelWheel

VelWheel1, -2 und -3 ist die Istgeschwindigkeit der einzelnen Motoren. Sie wird auch in $\frac{m}{s}$ angegeben. Die Berechnung erfolgt über den Encoder des Motors und den Radumfang mit folgender Formel:

$$\text{VelWheel1} = - \frac{n_{1,\text{fromMotor}}}{n_{\text{max,KlemmeDrehzahlMessung}}} \cdot \frac{n_{\text{max,MotorRPM}}}{60} \cdot \ddot{u} \cdot 2\pi \cdot r_{\text{Wheel}} \quad (3.1)$$

Dabei bezeichnen:

- $n_{1,\text{fromMotor}}$ – Rohwert der Drehzahlmessung an der Motorklemme (vorzeichenbehaftet).
- $n_{\text{max,KlemmeDrehzahlMessung}} = 10\,000$ – Wert, den die Klemme bei maximaler Motordrehzahl zurückliefert (Skalierungsreferenz der Messung).
- $n_{\text{max,MotorRPM}} = 3600$ rpm – maximale Motordrehzahl. Zusammen mit dem vorherigen Term wird der Rohwert auf eine tatsächliche Drehzahl in Umdrehungen pro Minute umgerechnet; die Division durch 60 wandelt diese in Umdrehungen pro Sekunde um.
- $\ddot{u} = 0,0625$ – Getriebeübersetzung zwischen Motor- und Raddrehzahl (Ausgangsdrehzahl = Motordrehzahl $\cdot \ddot{u}$).
- $r_{\text{Wheel}} = 40$ mm – Radradius. Über $2\pi \cdot r_{\text{Wheel}}$ wird die Raddrehzahl (in Umdrehungen/s) in eine Umfangsgeschwindigkeit umgerechnet.

Die Einheit von r_{Wheel} ist Millimeter, somit ist die Einheit von VelWheelmm/s.

3.2 Koordinatentransformation Roboter -> Motoren

3.3 Koordinatentransformation Welt -> Roboter

3.4 Positionsregler

3.5 Auswertung

3.5.1 Vektorfahrweise

3.5.2 Positionsfahrweise

4 Beobachter

4.1 Zielsetzung

- Warum wird ein Beobachter benötigt?
- Entwicklung eines Beobachters zur Schätzung von Zuständen, die nicht direkt messbar sind.
- Welche Zustände werden nicht gemessen?

4.2 Kinematisches Beobachtermodell

- Inverse Kinematik von ?? => Umrechnung Roboterkoordinaten
- Umrechnung in Weltkoordinaten (wichtig, dass es zyklisch abläuft, da Winkeländerung zu Änderung in x/y Berechnung führt)
- Resultat: Aus Motordaten und Anfangsposition kann die aktuelle Position im Raum berechnet werden

Umrechnung in Roboterkoordinaten: Die Winkelgeschwindigkeit des Roboters ist in Roboterkoordinaten, als auch Weltkoordinaten gleich definiert, weshalb hier keine Unterscheidung vorgenommen wird.

$$\begin{bmatrix} v_x^{body} \\ v_y^{body} \\ \omega \end{bmatrix} = - \begin{bmatrix} -\frac{\sqrt{3}}{3} & 0 & \frac{\sqrt{3}}{3} \\ \frac{1}{3} & -\frac{2}{3} & \frac{1}{3} \\ -\frac{1}{3R} & -\frac{1}{3R} & -\frac{1}{3R} \end{bmatrix} \begin{bmatrix} V_{\omega 1} \\ V_{\omega 2} \\ V_{\omega 3} \end{bmatrix} \quad (4.1)$$

Geschwindigkeit in Weltkoordinaten:

$$\begin{bmatrix} v_x^{world} \\ v_y^{world} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} v_x^{body} \\ v_y^{body} \end{bmatrix} \quad (4.2)$$

Integration zu Position im Raum:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \cdot \begin{bmatrix} v_x^{world} \\ v_y^{world} \\ \omega \end{bmatrix} \quad (4.3)$$

4.3 Beobachterstruktur

- Luneburger
- EKF
- Zu beiden Blockschaltbilder erstellen
- Vergleichen und aufzeigen, warum EKF keinen bedeutenden Vorteil gegenüber Luneburger hat (Kamerafehler ist immer ähnlich, daher reicht statischer Gain)

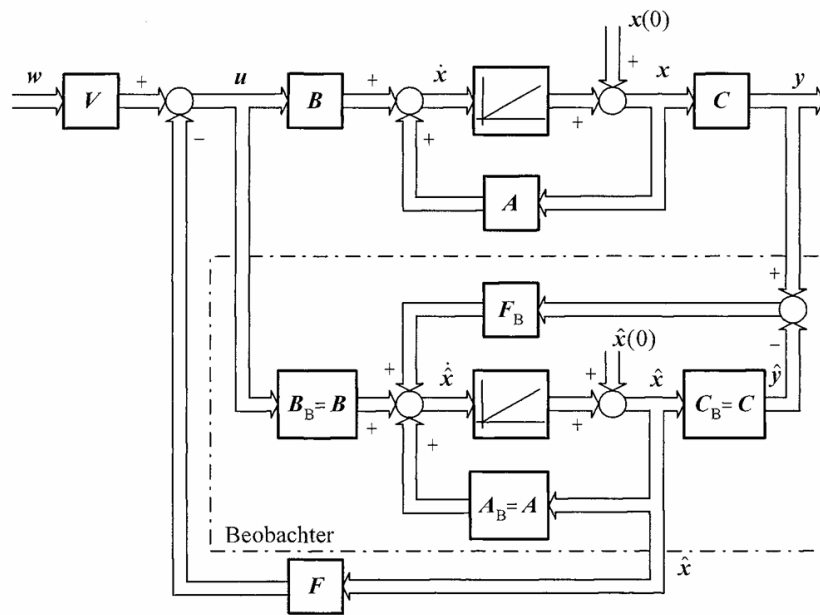


Abbildung 4.1: Zustandsbeobachter nach Luneburger [1]

4.4 Parametrierung

- Einstellen des Gains L
- Variabler Gain

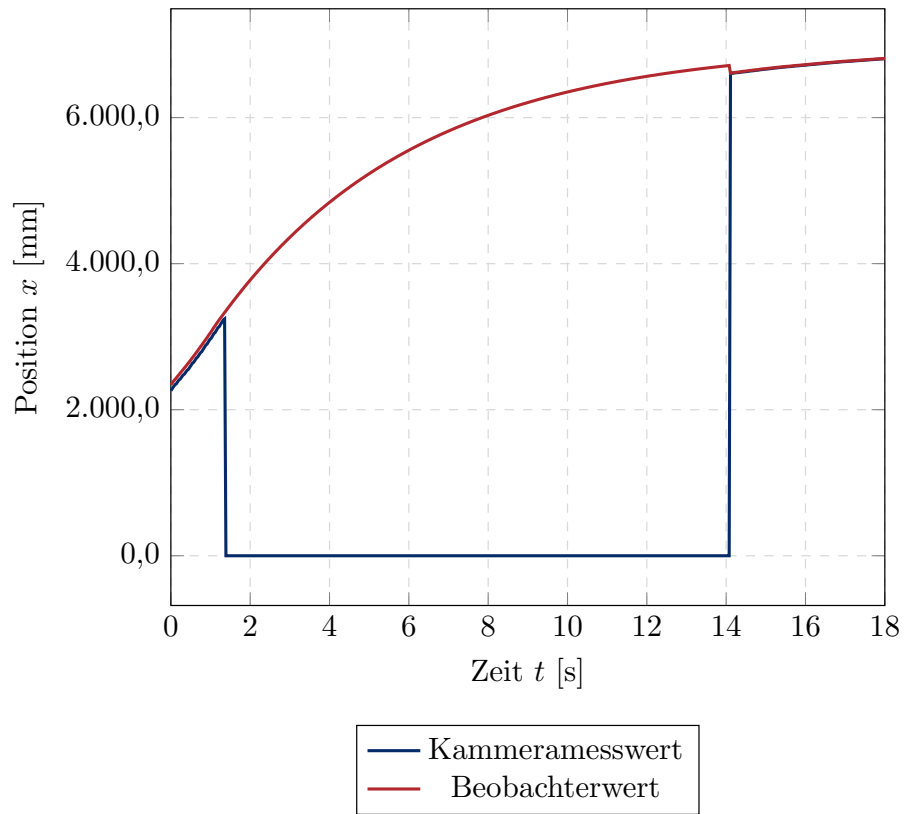


Abbildung 4.2: Beobachter mit direktem Ausgleich Gesamt

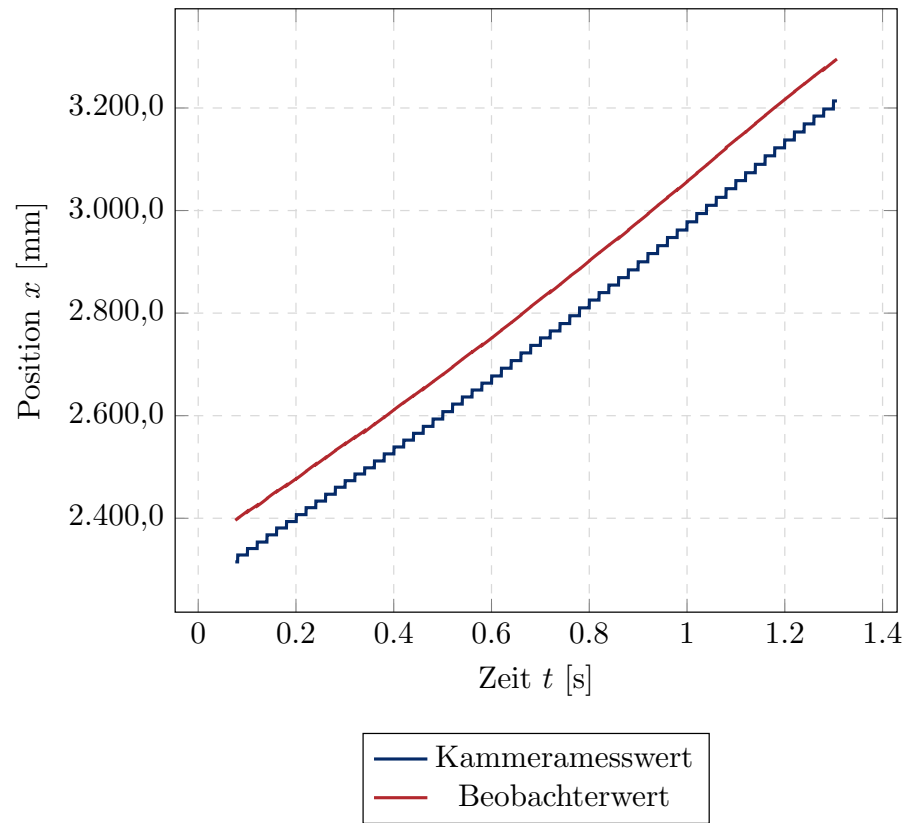


Abbildung 4.3: Beobachter mit direktem Ausgleich Normalfahrt

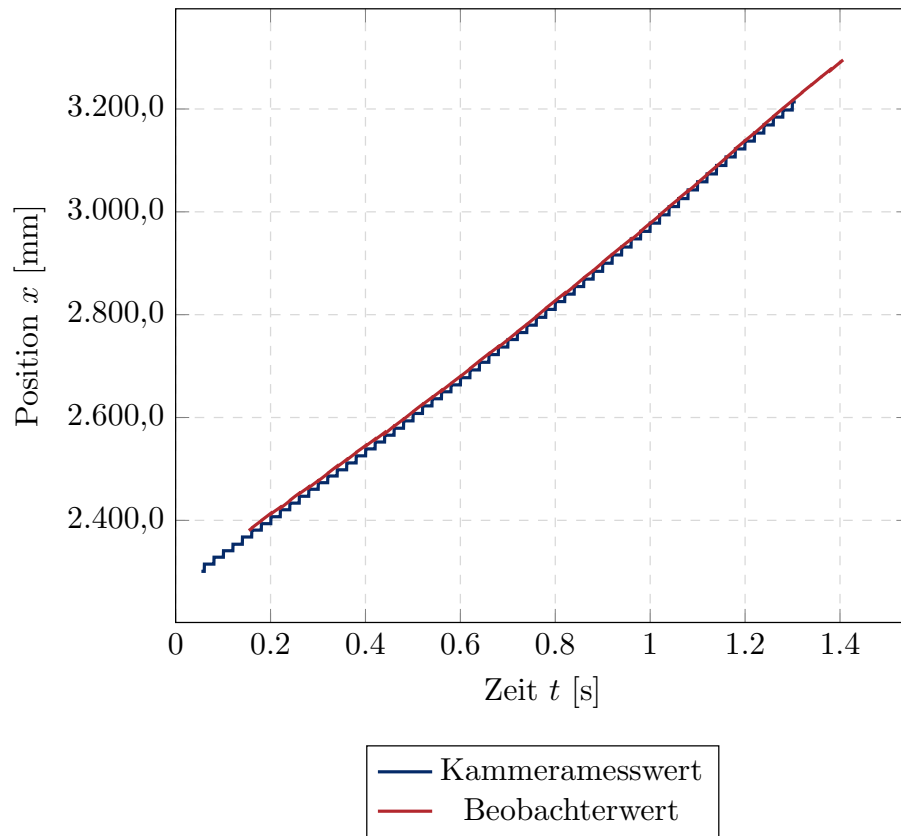


Abbildung 4.4: Beobachter mit direktem Ausgleich Normalfahrt Vershoben

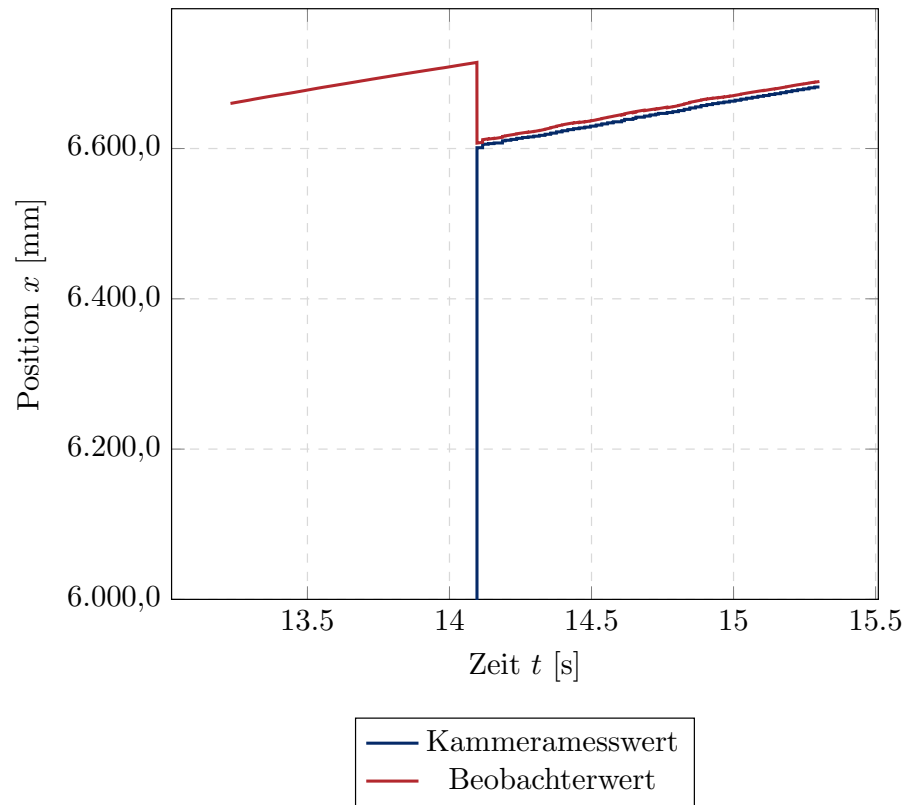


Abbildung 4.5: Beobachter mit direktem Ausgleich

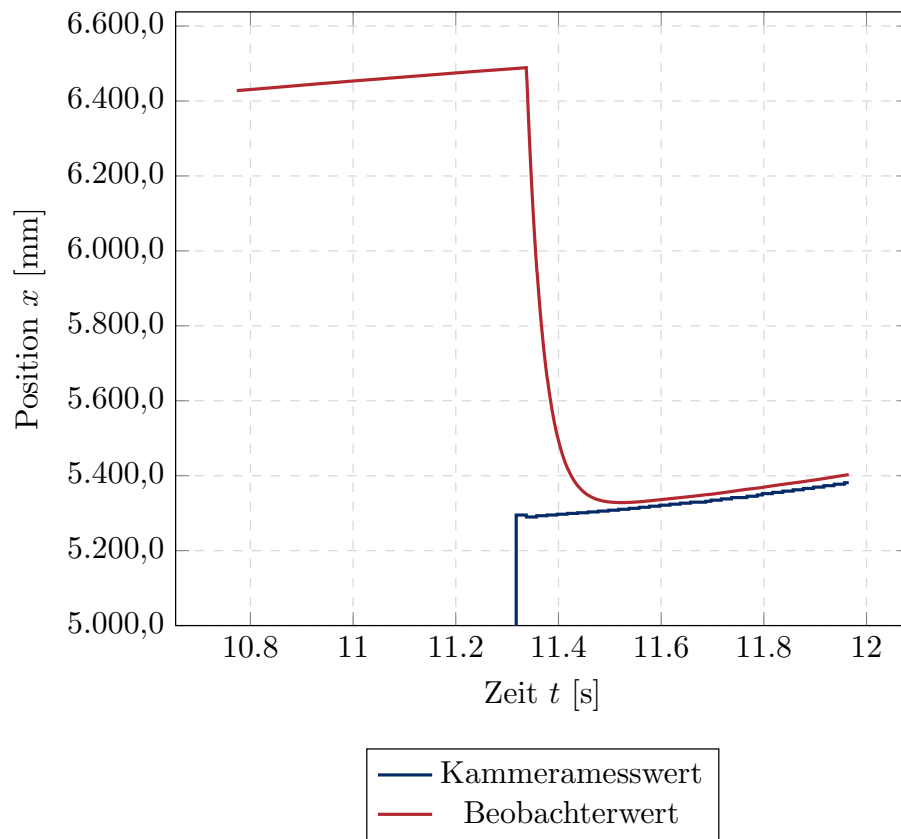


Abbildung 4.6: Beobachter mit langsamen Ausgleich

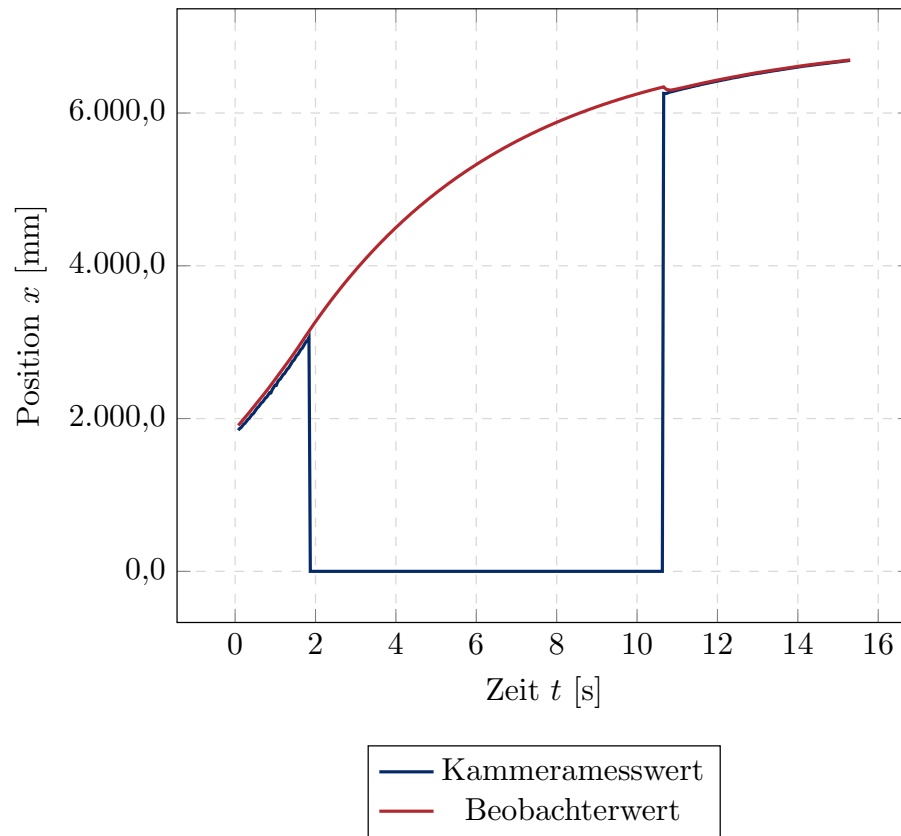


Abbildung 4.7: Beobachter mit variablen Gain

Is von der Idee her ganz nett, aber kein großer Effekt, da die Kamera, wenn Bild Erkannt, immer ähnlichen Fehler aufweist. Also lieber statischen Wert tunen.

Beobachter Code:

```
VAR_INPUT
  wheelSpeed : WheelVelocity;
  cameraData : CameraConnectionUDP;
  RobotIndex : INT;
END_VAR
VAR_OUTPUT
  eastPos : sPosition := (reX := 0, reY := 0, reWinkel := 0);
END_VAR
VAR CONSTANT
  // === BEOBACHTER-PARAMETER => Großer Gain bedeutet hohes vertrauen in Kamera ===
  L_Gain      : REAL := 1.0;    // Luenberger Gain für X    (0.0001-0.1)

  CYCLETIME    : LREAL := 0.00033333; // Cycletime des Motortasks
  CAMERA_DELAY : REAL  := 0.1;
  BUFFWE_LENGTH : INT  := 500;
  CAMERA_ERROR_THRESHOLD : REAL := 5.0; // Fehlergrenze
  CAMERA_TIMEOUT : LREAL := 30.0; // Timeout in Sekunden
END_VAR

VAR_STAT
  bufferWheelSpeed : ARRAY[0..BUFFWE_LENGTH] OF WheelVelocity; // Speichern der letzten
  prefCamPos       : sPosition; // vorherige Kamera Position
  camPos           : sPosition;
  modelPos         : sPosition;
  CurrentTime      : LREAL;
  CameraTimestamp  : LREAL;
  actualError      : LREAL; // Kamerafehler
  bufferIdx        : INT := 0;
  init             : BOOL;
END_VAR

// Am anfang einmal die Kammeraposition direkt übertragen
IF init = TRUE THEN
  // Wenn ein guter Kamerawert gefunden wurde direkt einmal übertragen
  IF cameraData.robotPosition[RobotIndex].e < CAMERA_ERROR_THRESHOLD THEN
    camPos.reX      := cameraData.robotPosition[RobotIndex].x;
    camPos.reY      := cameraData.robotPosition[RobotIndex].y;
    camPos.reWinkel := cameraData.robotPosition[RobotIndex].theta;
    actualError     := cameraData.robotPosition[RobotIndex].e;
    prefCamPos     := camPos;
    modelPos       := camPos;
    eastPos        := camPos;
```

```

    init := FALSE;
  END_IF
ELSE

  // Radwerte in FIFO abspeichern

  bufferWheelSpeed[bufferIdx] := wheelSpeed;
  bufferIdx := (bufferIdx + 1) MOD BUFFWE_LENGTH;

  // Modell Propagieren
  propagate_state(
    X_Start := modelPos,
    wheelSpeed := wheelSpeed,
    P_Start := P_currentState,
    deltaT := CYCLETIME,
    X_Predict => modelPos,
    P_Predict => P_currentState
  );
  CurrentTime := CurrentTime + CYCLETIME; // Zeit weiter laufen lassen, wenn keine

  // Kameradaten verarbeiten

  // Kammeraposition erst validieren
  IF cameraData.robotPosition[RobotIndex].e > CAMERA_ERROR_THRESHOLD OR
  // Prüfen ob neue Kameraposition
  ((prefCamPos.reX = cameraData.robotPosition[RobotIndex].x) AND
  (prefCamPos.reY = cameraData.robotPosition[RobotIndex].y) AND
  (prefCamPos.reWinkel = cameraData.robotPosition[RobotIndex].theta)) THEN

    // Falls keine neue Kameraposition erscheint, auch die Kamera vorwärtspropagieren
    propagate_state(
      X_Start := camPos,
      wheelSpeed := wheelSpeed,
      P_Start := P_currentState,
      deltaT := CYCLETIME,
      X_Predict => camPos,
      P_Predict => P_currentState
    );
  ELSE
    // Falls Kameraposition gut nehmen
    camPos.reX := cameraData.robotPosition[RobotIndex].x;
    camPos.reY := cameraData.robotPosition[RobotIndex].y;
    camPos.reWinkel := cameraData.robotPosition[RobotIndex].theta;
  
```

```

actualError    := cameraData.robotPosition[RobotIndex].e;
CameraTimestamp := (cameraData.HH * 60 + cameraData.MM) * 60 + cameraData.SS;
CurrentTime    := CameraTimestamp;

// Abspeichern der actualposition als alte Kameraposition

prefCamPos := camPos;

// Alle Werte zurückrechnen
steps := LREAL_TO_INT(CAMERA_DELAY / CYCLETIME);
startIdx := bufferIdx - steps;
IF startIdx < 0 THEN
    startIdx := startIdx + BUFFWE_LENGTH;
END_IF

FOR i := 0 TO steps - 1 DO
    idx := startIdx + i;
    IF idx >= BUFFWE_LENGTH THEN
        idx := idx - BUFFWE_LENGTH;
    END_IF

    propagate_state(
        X_Start    := camPos,
        wheelSpeed := bufferWheelSpeed[idx],
        P_Start    := P_currentState,
        deltaT     := CYCLETIME,
        X_Predict  => camPos,
        P_Predict  => P_currentState
    );
END_FOR
END_IF

// LUENBERGER-KORREKTUR:  $\hat{x} = x_{\text{model}} + L * (y - x_{\text{model}})$ 

// variableGain := L_Gain * actualError/CAMERA_ERROR_THRESHOLD;
// IF variableGain > 1 OR variableGain < 0.0001 THEN
//     variableGain := L_Gain;
// END_IF
// eastPos.reX    := modelPos.reX    + variableGain * (camPos.reX    - modelPos.reX);
// eastPos.reY    := modelPos.reY    + variableGain * (camPos.reY    - modelPos.reY);
// eastPos.reWinkel := modelPos.reWinkel + variableGain * NormalizeAngle(camPos.reWinkel);

eastPos.reX    := modelPos.reX    + L_Gain * (camPos.reX    - modelPos.reX);

```

```
eastPos.reY      := modelPos.reY      + L_Gain * (camPos.reY      - modelPos.reY);  
eastPos.reWinkel := modelPos.reWinkel + L_Gain *  NormalizeAngle(camPos.reWinkel);  
modelPos := eastPos;  
END_IF
```

GetMinDistance Funktion:

```
VAR_INPUT
    velAngle      : LREAL;
    tolerance     : LREAL;
END_VAR

FOR i := 1 TO 9 DO
    sensorAngle := (i - 1) * 40.0;

    diff := sensorAngle - velAngle;

    IF diff > 180.0 THEN
        diff := diff - 360.0;
    ELSIF diff < -180.0 THEN
        diff := diff + 360.0;
    END_IF;

    IF ABS(diff) <= tolerance THEN
        minDistance := MIN(minDistance, IrDistanceSensors.distance_mm[i]);
    END_IF;
END_FOR;

GetMinDistance := minDistance;
```

Normalize Angle Funktion:

```
VAR_INPUT
  angle : LREAL;
END_VAR
VAR_OUTPUT
  NormalizeAngle :LREAL;
END_VAR

WHILE angle > PI DO
  angle := angle - (2.0 * PI);
END_WHILE

WHILE angle < -PI DO
  angle := angle + (2.0 * PI);
END_WHILE

NormalizeAngle := angle;
```

Propagate State Funktion:

```
VAR_INPUT
  X_Start : sPosition;
  wheelSpeed : WheelVelocity;  (* w1,w2,w3 *)
  P_Start : ARRAY[1..9] OF LREAL;
  deltaT : LREAL;
END_VAR

VAR_OUTPUT
  X_Predict : sPosition;
  P_Predict : ARRAY[1..9] OF LREAL;
END_VAR

(* EKF PREDICTION STEP - 3 Wheel Omni Robot *)
(* Zustandsvektor: x = [posX, posY, yaw] *)
(* Modell: rein kinematisch *)
(* Diskretisierung: Vorwärts-Euler *)
(* ===== *)

X_Start.reWinkel := -X_Start.reWinkel;

(* --- Wheel -> Body --- *)
bodyVelX := -(2.0/3.0) * (
  -SIN(60 * PI/180) * wheelSpeed.wheel1
  -SIN(180 * PI/180) * wheelSpeed.wheel2
  -SIN(300 * PI/180) * wheelSpeed.wheel3
);

bodyVelY := -(2.0/3.0) * (
  COS(60 * PI/180) * wheelSpeed.wheel1
  + COS(180 * PI/180) * wheelSpeed.wheel2
  + COS(300 * PI/180) * wheelSpeed.wheel3
);

yawRate := -(wheelSpeed.wheel1 + wheelSpeed.wheel2 + wheelSpeed.wheel3)
  / (3.0 * GVL_RobotConstant.ROBOT_RADIUS * 1000); // Umrechnung in mm

(* --- Body -> World --- *)
worldVelX := bodyVelX * COS(X_Start.reWinkel)
  - bodyVelY * SIN(X_Start.reWinkel);

worldVelY := bodyVelX * SIN(X_Start.reWinkel)
```

```

    + bodyVelY * COS(X_Start.reWinkel);

(* --- Integration --- *)
X_Predict.reX    := X_Start.reX + deltaT * worldVelX;
X_Predict.reY    := X_Start.reY + deltaT * worldVelY;
X_Predict.reWinkel := X_Start.reWinkel + deltaT * yawRate;

(* --- Winkel normalisieren --- *)

NormalizeAngle(angle := X_Predict.reWinkel, NormalizeAngle => X_Predict.reWinkel);

X_Predict.reWinkel := - X_Predict.reWinkel;

P_Predict := P_Start;

```

5 Ausblick: Fuzzy Logic Regler

Zielsetzung:

- Ein Regler für alles
- Vectorfahrt performanter gestalten (Punktliste erzeugen)
- Erfahrung in unterschiedlichen Reglerstrukturen sammeln

5.1 Konzept

5.2 Parametertuning durch Simulation

5.3 Parameteranpassung am realen System

6 Fazit

Abbildungsverzeichnis

3.1	Drehzahlregler als Blackbox mit Eingangs- und Ausgangssignalen . .	5
4.1	Zustandsbeobachter nach Luneburger [1]	8
4.2	Beobachter mit direktem Ausgleich Gesamt	9
4.3	Beobachter mit direktem Ausgleich Normalfahrt	10
4.4	Beobachter mit direktem Ausgleich Normalfahrt Vershoben	11
4.5	Beobachter mit direktem Ausgleich	12
4.6	Beobachter mit langsamen Ausgleich	13
4.7	Beobachter mit variablen Gain	14

Tabellenverzeichnis

1 Konventionen vi

Literatur

- [1] Jan Lunze. *Regelungstechnik 1: Systemtheoretische Grundlagen, Analyse und Entwurf einschleifiger Regelungen*. 10. Aufl. Springer Vieweg, 2016. ISBN: 978-3-642-29532-3.